

Relations and Their Properties

A 1.5-2 hour lecture on ordered pairs, relations, representations, properties, equivalence and CS applications.

- Recommended teaching duration: approximately 1.5-2 hours including board work, worked examples, Python demonstrations and student discussion.
- Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

- Concept introduction and terminology
- Detailed definitions and notation
- Worked mathematical examples
- CS case studies with complete solutions
- Python implementation and verification
- Extended question bank
- 2-3 unsolved student practice questions at the end

1. Ordered Pairs and Cartesian Products

An ordered pair (a,b) records a first and second component. A relation from A to B is a subset of $A \times B$.

2. Relation on a Set

A relation on A is a subset of $A \times A$. It may contain self-pairs, reverse pairs, or directed connections depending on the application.

3. Domain and Range of a Relation

The domain consists of first components appearing in the relation; the range consists of second components.

4. Reflexive

R is reflexive on A when $(a,a) \in R$ for every $a \in A$.

5. Symmetric and Antisymmetric

Symmetric means $(a,b) \in R$ implies $(b,a) \in R$. Antisymmetric means if both (a,b) and (b,a) occur, then $a=b$.

6. Transitive

If $(a,b) \in R$ and $(b,c) \in R$, then (a,c) must be in R . Transitivity is important in dependency and ordering systems.

7. Equivalence Relations

Reflexive + symmetric + transitive gives an equivalence relation. Equivalence relations partition a set into equivalence classes.

8. Relation Representations

Use ordered pairs, tables, directed graphs, or matrices. A Boolean matrix contains 1 when a relation pair exists and 0 otherwise.

9. CS Case Study — Social Network

R contains (u,v) when u follows v. This relation is generally not symmetric: u may follow v without v following u.

Solution: test a pair and its reverse. A mutual-follow relation is symmetric, while a one-way follower relation is not.

10. CS Case Study — Course Prerequisites

R contains (A,B) when course A is a prerequisite of B. If Programming→DataStructures and DataStructures→Algorithms, a curriculum model may also need Programming→Algorithms.

Solution: the transitive closure captures implied prerequisite paths and prevents incomplete dependency information.

11. CS Case Study — Same Role

Define uRv when users u and v have exactly the same role set. This relation is reflexive, symmetric and transitive, so it is an equivalence relation.

12. Python Relation Checking

Store a relation as a set of tuples and use membership tests to check properties.

Python Laboratory

Python Activity 1

```
A={'Ali', 'Sara', 'Hamza'}  
R={('Ali', 'Sara'), ('Sara', 'Ali'), ('Ali', 'Hamza')}
```

```
print(('Hamza', 'Ali') in R) # symmetry test for Ali-Hamza
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
def is_symmetric(R):  
    return all((b,a) in R for (a,b) in R)
```

```
print(is_symmetric(R))
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 3

```
def is_transitive(R):  
    for a,b in R:  
        for c,d in R:  
            if b == c and (a,d) not in R:  
                return False  
    return True
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5-2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Write a relation as a set of ordered pairs.
2. Find the domain and range of a relation.
3. Test reflexivity for a relation on a finite set.
4. Test symmetry and antisymmetry.
5. Test transitivity.
6. Determine whether a relation is an equivalence relation.
7. Represent a relation as a matrix.
8. Model followers in a social network.
9. Model course prerequisites.
10. Model “same department” as a relation on employees.
11. Write Python to test symmetry.
12. Write Python to test transitivity.
13. Explain why a prerequisite relation should normally be directed.

End-of-Lecture Student Practice — No Solutions

1. Create a relation for a small social network and test all four properties.
2. Create a prerequisite relation and test transitivity.
3. Represent one relation as a matrix.

Quick Recap

Relations model connections, dependencies and classifications.